



Chapter 2: Node.js

Table of Contents

- [2.1 What is Node.js?](#)
 - [2.2 Event-Driven Architecture](#)
 - [2.3 Non-Blocking I/O](#)
 - [2.4 Core Modules](#)
 - [2.5 NPM \(Node Package Manager\)](#)
 - [2.6 Environment Variables](#)
 - [2.7 Streams & Buffers](#)
-

2.1 What is Node.js?

Definition: Node.js is a JavaScript runtime built on Chrome's V8 engine that allows running JavaScript on the server-side. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for building scalable network applications.

Key Features:

- Single-threaded with event loop
- Non-blocking asynchronous I/O
- NPM - World's largest package ecosystem
- Cross-platform

```
// Simple Node.js HTTP server
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js!');
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

2.2 Event-Driven Architecture

Definition: In event-driven architecture, the flow of the program is determined by events (user actions, messages, sensor outputs). Node.js uses an EventEmitter class to handle events, allowing asynchronous handling of multiple concurrent operations.

```
const EventEmitter = require('events');

// Create emitter instance
const emitter = new EventEmitter();

// Register event listener
```

```

emitter.on('orderPlaced', (order) => {
  console.log('Order received:', order);
});

// Register another listener for same event
emitter.on('orderPlaced', (order) => {
  console.log('Sending email for order:', order.id);
});

// Once listener (fires only once)
emitter.once('firstVisit', (user) => {
  console.log('Welcome, new user:', user);
});

// Emit events
emitter.emit('orderPlaced', { id: 1, item: 'Book', qty: 2 });
emitter.emit('firstVisit', 'John');
emitter.emit('firstVisit', 'Jane'); // Won't trigger (once)

// Remove listener
const callback = (data) => console.log(data);
emitter.on('test', callback);
emitter.removeListener('test', callback);

// Error handling
emitter.on('error', (err) => {
  console.error('Error occurred:', err.message);
});

// Custom class extending EventEmitter
class OrderService extends EventEmitter {
  placeOrder(order) {
    // Process order...
    this.emit('orderPlaced', order);
    this.emit('notify', `Order ${order.id} placed`);
  }
}

const orderService = new OrderService();
orderService.on('orderPlaced', (order) => console.log('Processing:', order));
orderService.placeOrder({ id: 123, item: 'Laptop' });

```

2.3 Non-Blocking I/O

Definition: Non-blocking I/O means that I/O operations (file reading, network requests, database queries) don't block the execution of other code. Node.js can handle thousands of concurrent connections because it doesn't wait for operations to complete before moving on to the next task.

```

const fs = require('fs');

```

```
// ===== BLOCKING (Synchronous) =====
console.log('Start');
const data = fs.readFileSync('file.txt', 'utf8'); // BLOCKS here
console.log(data);
console.log('End');
// Output: Start → [file content] → End (sequential)

// ===== NON-BLOCKING (Asynchronous) =====
console.log('Start');
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data); // Runs later when file is read
});
console.log('End');
// Output: Start → End → [file content] (non-blocking)

// ===== Promise-based (Modern approach) =====
const fsPromises = require('fs').promises;

async function readFileAsync() {
  console.log('Start');
  const data = await fsPromises.readFile('file.txt', 'utf8');
  console.log(data);
  console.log('End');
}

// Why non-blocking matters:
// With blocking I/O:
// Request 1 → Wait 100ms → Done
// Request 2 → Wait 100ms → Done
// Request 3 → Wait 100ms → Done
// Total: 300ms

// With non-blocking I/O:
// Request 1 → |
// Request 2 → | → All waiting simultaneously
// Request 3 → |
// Total: ~100ms
```

2.4 Core Modules

Definition: Node.js comes with built-in modules that provide essential functionality without external installation. Common modules include `fs` (file system), `path` (file paths), `http` (web server), `events` (event handling), and `os` (operating system info).

File System (fs)

```
const fs = require('fs');
const fsPromises = require('fs').promises;
```

```

// ===== READ FILE =====
// Sync
const content = fs.readFileSync('file.txt', 'utf8');

// Async callback
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Async/await
const data = await fsPromises.readFile('file.txt', 'utf8');

// ===== WRITE FILE =====
fs.writeFileSync('output.txt', 'Hello World');
await fsPromises.writeFile('output.txt', 'Hello World');

// ===== APPEND TO FILE =====
fs.appendFileSync('log.txt', 'New entry\n');

// ===== CHECK IF EXISTS =====
if (fs.existsSync('file.txt')) {
  console.log('File exists');
}

// ===== DELETE FILE =====
fs.unlinkSync('file.txt');
await fsPromises.unlink('file.txt');

// ===== CREATE DIRECTORY =====
fs.mkdirSync('new-folder', { recursive: true });

// ===== READ DIRECTORY =====
const files = fs.readdirSync('.');
console.log(files); // ['file1.txt', 'file2.txt', ...]

// ===== FILE STATS =====
const stats = fs.statSync('file.txt');
console.log(stats.isFile()); // true
console.log(stats.isDirectory()); // false
console.log(stats.size); // bytes

```

Path Module

```

const path = require('path');

// Join paths (handles separators)
path.join('/users', 'john', 'documents', 'file.txt');
// '/users/john/documents/file.txt'

// Resolve to absolute path

```

```

path.resolve('file.txt');
// '/current/working/directory/file.txt'

// Get filename
path.basename('/users/john/file.txt'); // 'file.txt'
path.basename('/users/john/file.txt', '.txt'); // 'file'

// Get extension
path.extname('file.txt'); // '.txt'
path.extname('file.config.js'); // '.js'

// Get directory
path.dirname('/users/john/file.txt'); // '/users/john'

// Parse path
path.parse('/users/john/file.txt');
// { root: '/', dir: '/users/john', base: 'file.txt', ext: '.txt', name: 'file' }

// Current file and directory (ES modules)
import.meta.url; // File URL
// For CommonJS
__filename; // Current file path
__dirname; // Current directory path

```

HTTP Module

```

const http = require('http');

// Create server
const server = http.createServer((req, res) => {
  // Request info
  console.log(req.method); // 'GET', 'POST', etc.
  console.log(req.url); // '/api/users'
  console.log(req.headers); // { 'content-type': '...', ... }

  // Routing
  if (req.method === 'GET' && req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html' });
    res.end('<h1>Home Page</h1>');
  } else if (req.method === 'GET' && req.url === '/api/users') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify([ { id: 1, name: 'John' } ]));
  } else if (req.method === 'POST' && req.url === '/api/users') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', () => {
      const user = JSON.parse(body);
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify(user));
    });
  } else {

```

```

        res.writeHead(404, { 'Content-Type': 'text/plain' });
        res.end('Not Found');
    }
});

server.listen(3000, () => console.log('Server on port 3000'));

// Make HTTP request (client)
http.get('http://api.example.com/data', (res) => {
    let data = '';
    res.on('data', chunk => { data += chunk; });
    res.on('end', () => console.log(JSON.parse(data)));
});

```

OS Module

```

const os = require('os');

os.platform();    // 'darwin', 'win32', 'linux'
os.arch();        // 'x64', 'arm64'
os.cpus();        // CPU info array
os.cpus().length; // Number of cores
os.freemem();     // Free memory (bytes)
os.totalmem();    // Total memory (bytes)
os.homedir();     // User home directory
os.hostname();    // Computer name
os.uptime();      // System uptime (seconds)
os.tmpdir();      // Temp directory path

```

2.5 NPM (Node Package Manager)

Definition: NPM is the world's largest software registry and the default package manager for Node.js. It allows developers to share and reuse code packages, manage project dependencies, run scripts, and handle versioning.

```

# Initialize new project
npm init          # Interactive
npm init -y       # Skip questions (defaults)

# Install packages
npm install express      # Add to dependencies
npm install nodemon -D   # Add to devDependencies
npm install -g typescript # Install globally

# Shorthand
npm i express
npm i -D jest

# Install specific version
npm i express@4.18.2

```

```
npm i express@latest

# Install from package.json
npm install      # or just: npm i

# Uninstall
npm uninstall express
npm un express   # shorthand

# Update packages
npm update        # Update all
npm update express # Update specific
npm outdated      # Check outdated packages

# View package info
npm view express
npm view express versions

# List installed packages
npm list          # All
npm list --depth=0 # Top level only
npm list -g       # Global packages

# Run scripts (from package.json)
npm start
npm test
npm run dev      # Custom scripts need 'run'

# NPM cache
npm cache clean --force
```

package.json structure:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "My MERN application",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest",
    "build": "webpack --mode production"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "jest": "^29.0.0"
  }
}
```

```
  },
  "engines": {
    "node": ">=18.0.0"
  }
}
```

Version symbols:

- `^4.18.2` - Compatible (allows minor & patch updates: 4.x.x)
- `~4.18.2` - Approximately (allows patch updates: 4.18.x)
- `4.18.2` - Exact version only

2.6 Environment Variables

Definition: Environment variables are external configuration values stored outside the codebase. They're used for sensitive data (API keys, database URLs, secrets) and environment-specific settings, keeping configuration separate from code and secure.

```
// .env file (DO NOT commit to git!)
// PORT=3000
// MONGODB_URI=mongodb://localhost:27017/myapp
// JWT_SECRET=your-super-secret-key
// NODE_ENV=development

// Install dotenv
// npm install dotenv

// Load at the TOP of entry file
require('dotenv').config();

// Access variables
const port = process.env.PORT || 3000;
const mongoUri = process.env.MONGODB_URI;
const jwtSecret = process.env.JWT_SECRET;
const isProduction = process.env.NODE_ENV === 'production';

// Best practices
// 1. Create .env.example with placeholder values
// 2. Add .env to .gitignore
// 3. Validate required variables on startup

function validateEnv() {
  const required = ['MONGODB_URI', 'JWT_SECRET'];
  const missing = required.filter(key => !process.env[key]);

  if (missing.length > 0) {
    throw new Error(`Missing env variables: ${missing.join(', ')}`);
  }
}
validateEnv();
```

```
// Config module pattern
// config.js
module.exports = {
  port: process.env.PORT || 3000,
  mongoUri: process.env.MONGODB_URI,
  jwtSecret: process.env.JWT_SECRET,
  jwtExpire: process.env.JWT_EXPIRE || '7d',
  isProduction: process.env.NODE_ENV === 'production'
};

// Usage
const config = require('./config');
app.listen(config.port);
```

2.7 Streams & Buffers

Definition: Streams are objects that let you read/write data piece by piece (chunks) instead of loading everything into memory. Buffers are temporary storage for binary data. Streams are essential for handling large files or real-time data efficiently.

```
const fs = require('fs');

// ===== BUFFERS =====
// Create buffer
const buf1 = Buffer.alloc(10); // 10 bytes of zeros
const buf2 = Buffer.from('Hello'); // From string
const buf3 = Buffer.from([1, 2, 3, 4]); // From array

// Buffer operations
console.log(buf2.toString()); // 'Hello'
console.log(buf2.length); // 5 bytes
console.log(buf2.toJSON()); // { type: 'Buffer', data: [...] }

// ===== STREAMS =====
// Types:
// - Readable: Source of data (fs.createReadStream, http request)
// - Writable: Destination for data (fs.createWriteStream, http response)
// - Duplex: Both readable and writable (net.Socket)
// - Transform: Modify data as it passes through (zlib.createGzip)

// Reading large file with stream
const readStream = fs.createReadStream('large-file.txt', {
  encoding: 'utf8',
  highWaterMark: 64 * 1024 // 64KB chunks
});

readStream.on('data', (chunk) => {
  console.log(`Received ${chunk.length} bytes`);
});

readStream.on('end', () => console.log('Done reading'));
```

```
readStream.on('error', (err) => console.error(err));

// Writing with stream
const writeStream = fs.createWriteStream('output.txt');
writeStream.write('Hello ');
writeStream.write('World!');
writeStream.end(); // Close the stream

// Piping streams (most efficient)
const source = fs.createReadStream('input.txt');
const destination = fs.createWriteStream('output.txt');

source.pipe(destination);

// Piping with transform (compression)
const zlib = require('zlib');
const gzip = zlib.createGzip();

fs.createReadStream('file.txt')
  .pipe(gzip)
  .pipe(fs.createWriteStream('file.txt.gz'));

// HTTP streaming response
const http = require('http');
http.createServer((req, res) => {
  const stream = fs.createReadStream('large-video.mp4');
  stream.pipe(res);
}).listen(3000);
```

Next: [Chapter 3 - Express.js](#)